Can a Neural Network Learn the Laws of Nature?

What Happens When the Data Are Insufficient?

Alexander Heinlein

Nationale Wiskunde Dagen 2026, Universiteit Utrecht, March 27–28, 2026

Delft University of Technology

Outline

1. Workshop Outline
2. Neural Networks
3. The RC Circuit Model
4. From Data Fitting to Physics-Informed Learning

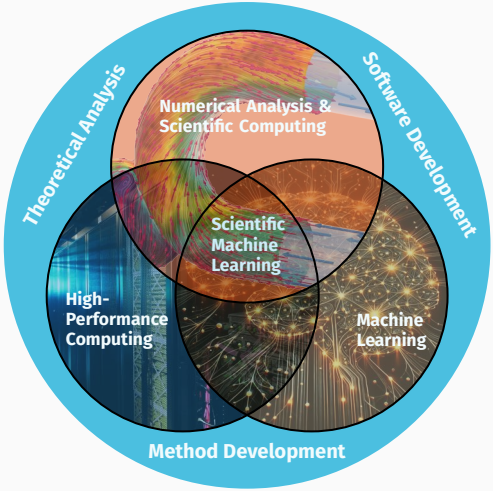
1. Workshop Outline

Alexander Heinlein

2016	**Ph. D.** Mathematics, U Cologne
2018 – 2020	**Managing Coordinator**, Center for Data and Simulation Science, U Cologne
2020 – 2021	**Acting Full Professor Numerical Methods for HPC**, U Stuttgart
Since 2021	**Assistant Professor Numerical Analysis**, TU Delft

Education at TU Delft

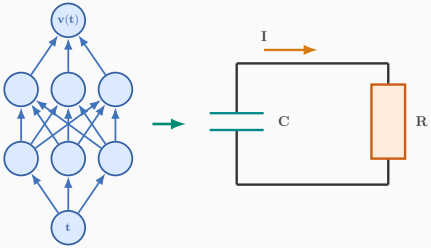
- Advanced Reservoir Simulation
- Computational Fluid Dynamics
- Fundamentals of AI
- Linear Algebra & Linear Algebra 2
- Linear Algebra and Optimization for Machine Learning
- Mathematical Data Science
- Scientific Machine Learning
- Special Topics in Computational Science and Engineering
- ...



Workshop Question and Path

Can a computer learn how a physical system behaves from **sparse** or **noisy** data?

- We first study what **neural networks** are and how they work.
- Then we test them on a **physical problem**: a **resistor-capacitor (RC) circuit**.
- For both parts, we also look at some **Python code** that you can test yourself.



Background Knowledge

- Basics of linear algebra,
- Basics of calculus,
- Circuit physics will be recapped,
- Reading basic Python code is helpful, but not required.

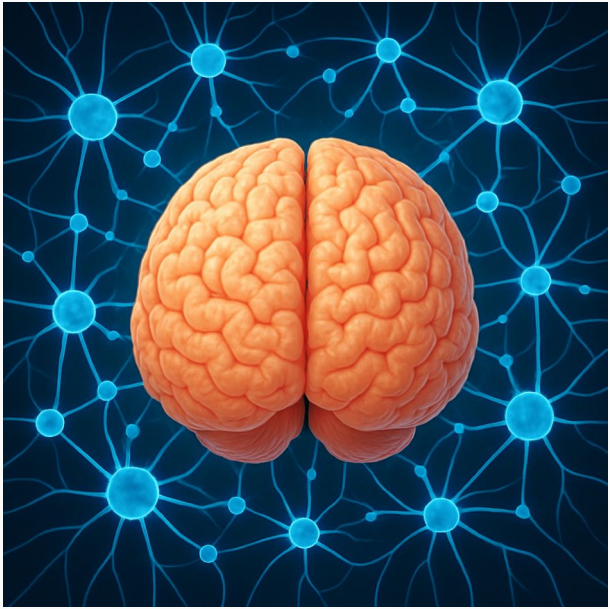
Learning Goals

By the end, you should be able to:

- Describe what a **neural network** is and how it approximates complex functions,
- Solve the **RC-circuit equation**,
- Explain why **data + physics** can outperform data alone.

2. Neural Networks

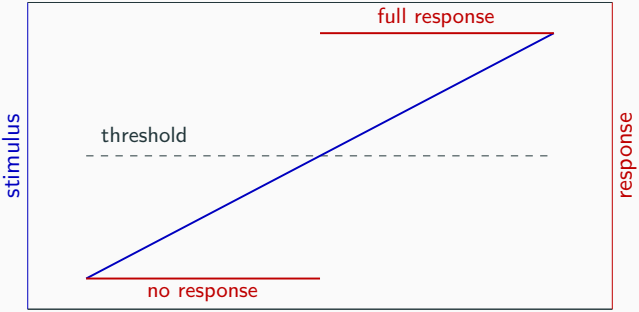
Artificial Intelligence and Neural Networks

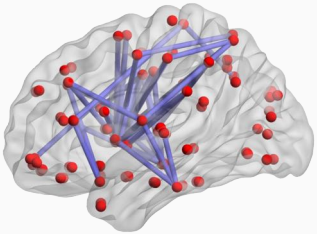


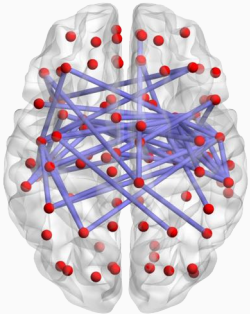
- Modern AI uses **artificial neural networks**, built on mathematics: matrix and vector operations.
- **Hopfield** and **Hinton** won the **2024 Nobel Prize in Physics** for pioneering artificial neural network models; they employ **principles of physics** to motivate the machine learning models.

Artificial Intelligence and Neural Networks

- **Artificial neural networks (ANNs)**, or simply **neural networks (NNs)**, are machine learning models loosely inspired by the **brain**.[a]
- A **biological neuron** responds only when the incoming signal is strong enough.







[a] W. S. McCulloch and W. Pitts, "A logical calculus of the ideas immanent in nervous activity," *Bulletin of Mathematical Biophysics*, 1943; F. Rosenblatt, "The perceptron: A probabilistic model for information storage and organization in the brain," *Psychological Review*, 1958.

Machine Learning

Wikipedia

"Machine learning (ML) is a field of study in artificial intelligence concerned with the development and study of statistical algorithms that ***can learn from data*** *and generalise to unseen data, and thus perform tasks* ***without explicit instructions****."*

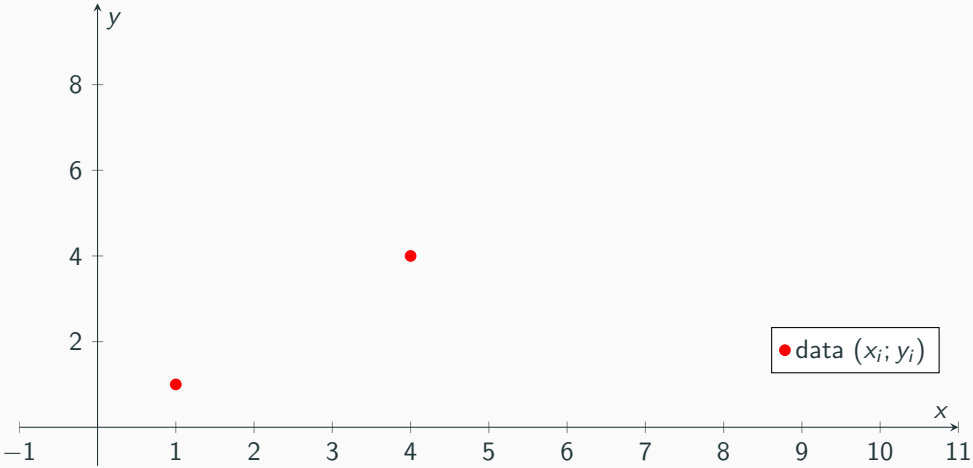
https://en.wikipedia.org/wiki/Machine_learning

Mitchell (1997)

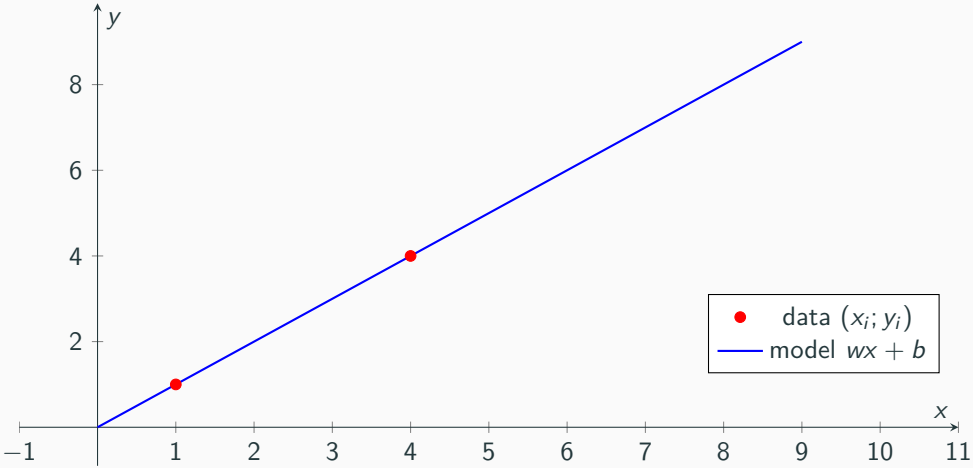
"A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P if its performance at tasks in T, as measured by P, improves with experience E."

— Mitchell, T. (1997). *Machine Learning*

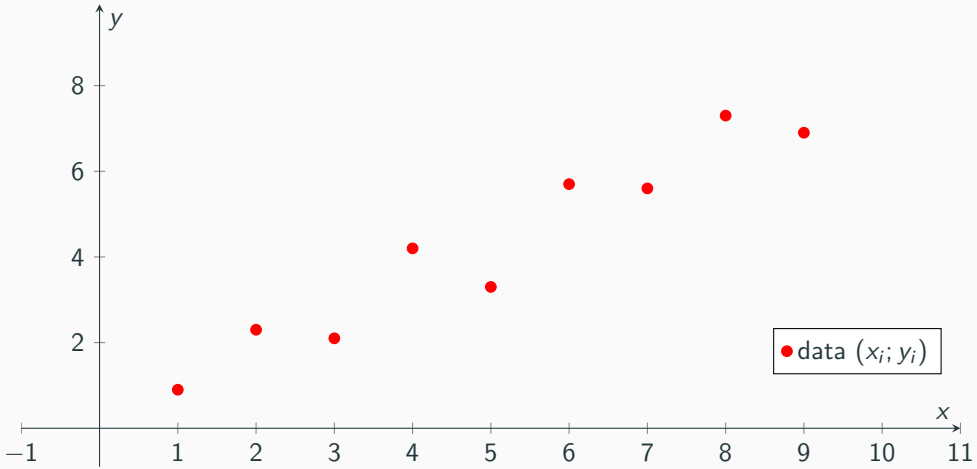
Learning From Data



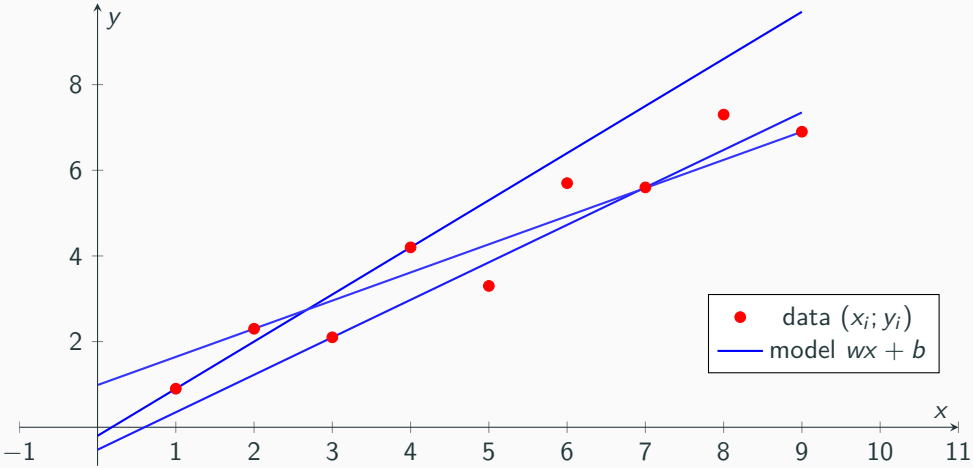
Learning From Data



Learning From Data

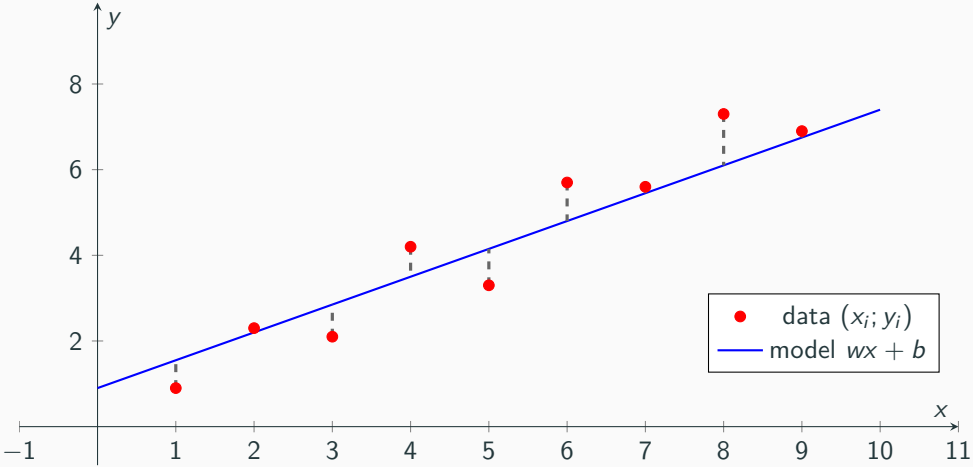


Learning From Data



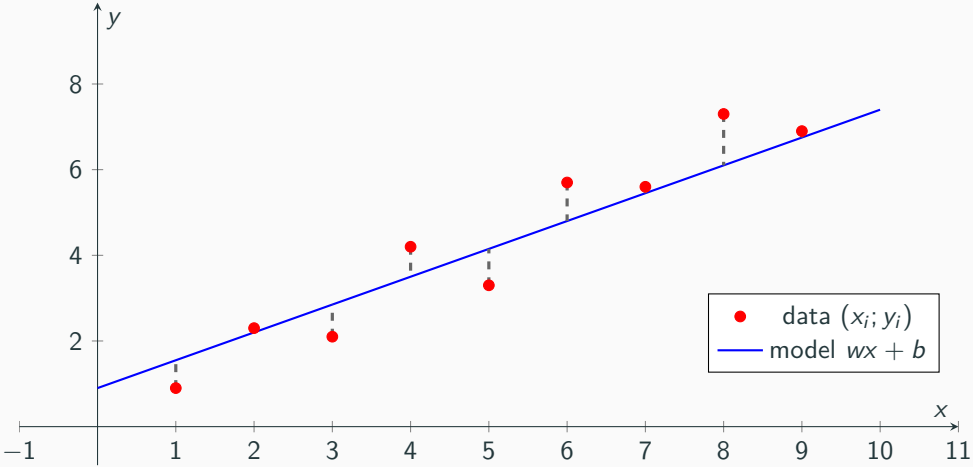
?

Linear Regression Fit



$$\text{Minimize} \quad \sum_{i=1}^{n} (y_i - (wx_i + b))^2$$

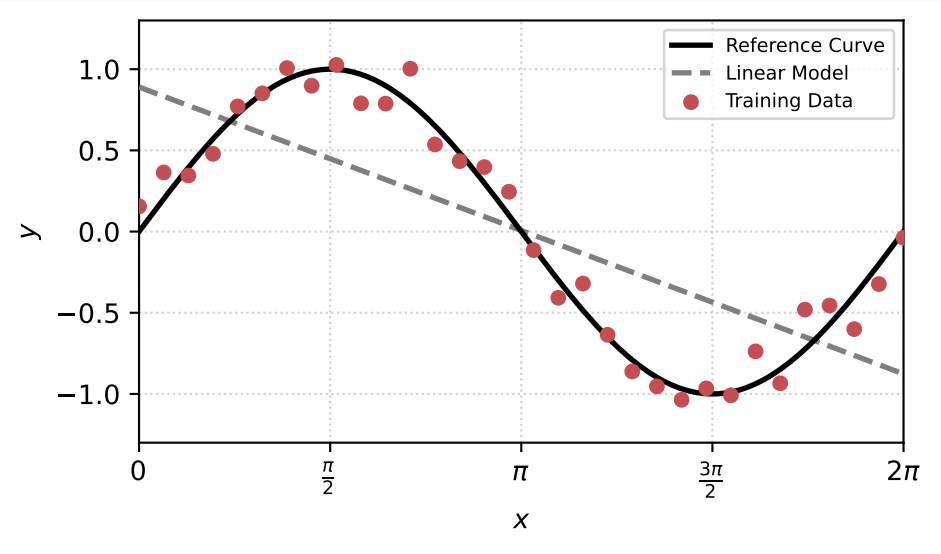
Linear Regression Fit



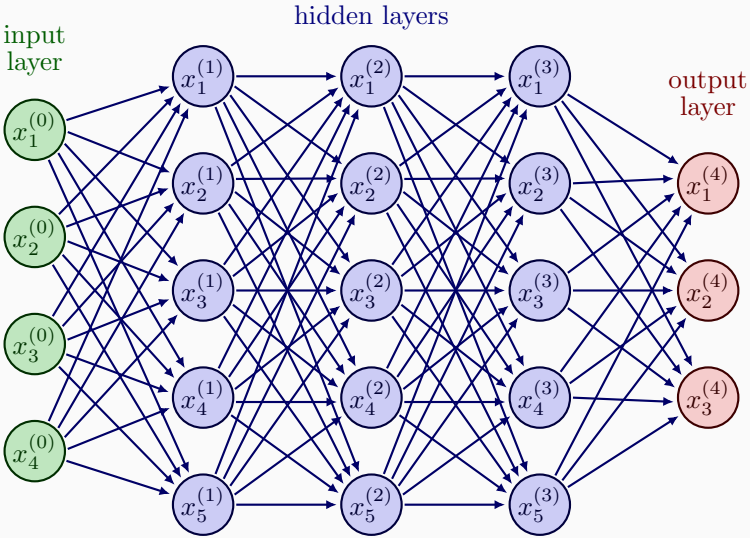
$$\text{Minimize} \quad \sum_{i=1}^{n} (y_i - (\mathbf{w}x_i + \mathbf{b}))^2$$

"learnable parameters"

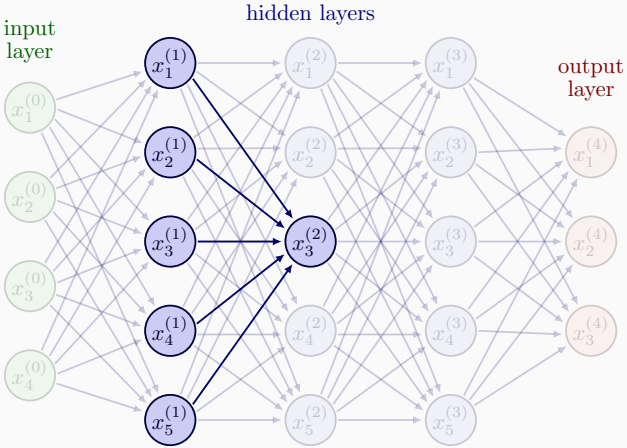
When Linear Regression Is Not Enough



Structure of an Artificial Neural Network Model (Part I)

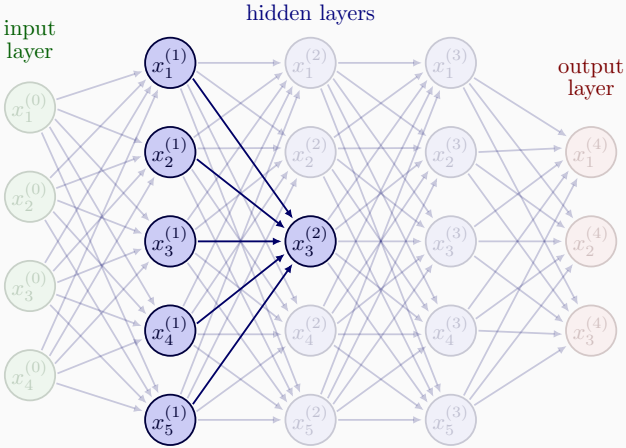


Structure of an Artificial Neural Network Model (Part I)



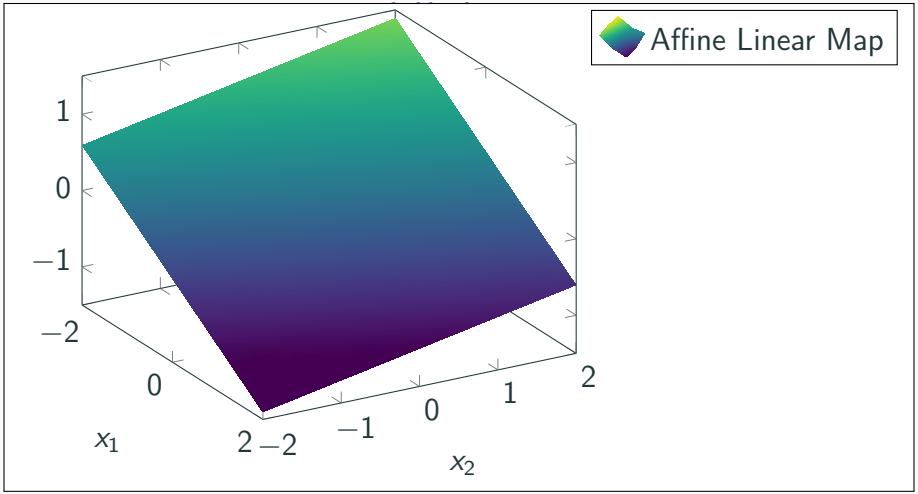
$$x_3^{(2)} = \Big(\sum_{i=1}^{5} w_i^{(2)} x_i^{(1)} + b_3^{(2)} \Big)$$

Structure of an Artificial Neural Network Model (Part I)



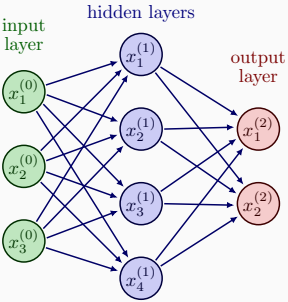
$$x_3^{(2)} = \sum_{i=1}^{5} \underbrace{w_i^{(2)}}_{\text{weight}} x_i^{(1)} + \underbrace{b_3^{(2)}}_{\text{bias}}$$

Structure of an Artificial Neural Network Model (Part I)



$$x_3^{(2)} = \sum_{i=1}^{5} \underbrace{w_i^{(2)}}_{\text{weight}} x_i^{(1)} + \underbrace{b_3^{(2)}}_{\text{bias}}$$

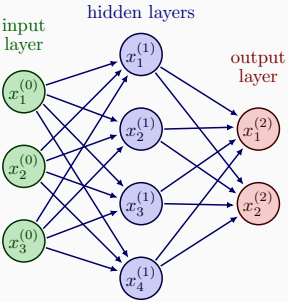
Composition of Affine Linear Transformations



If we **remove the activation** σ, the model is given by:

$$x^{(2)} = W^{(2)}x^{(1)} + b^{(2)} \qquad x^{(1)} = W^{(1)}x^{(0)} + b^{(1)}$$

Composition of Affine Linear Transformations



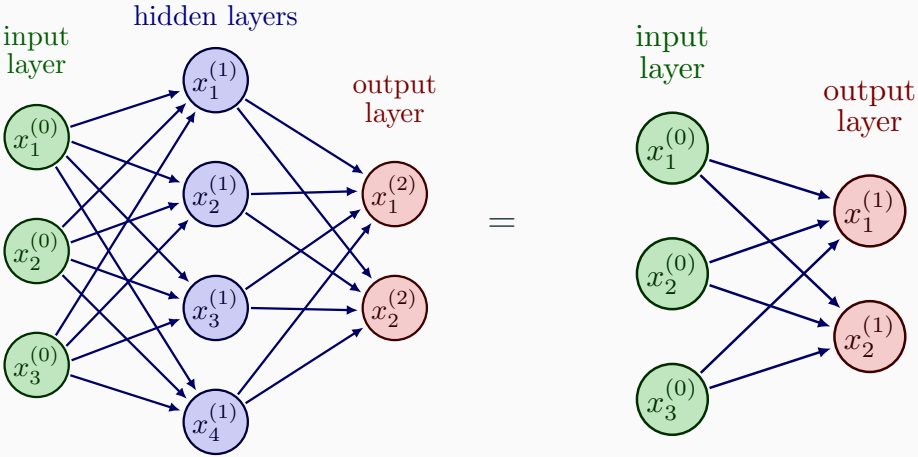
If we **remove the activation** σ, the model is given by:

$$x^{(2)} = W^{(2)}x^{(1)} + b^{(2)} \qquad x^{(1)} = W^{(1)}x^{(0)} + b^{(1)}$$

We obtain an **affine linear model**:

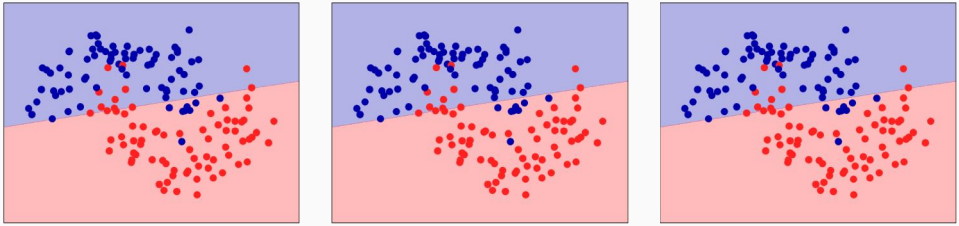
$$x^{(2)} = W^{(2)}\left(W^{(1)}x^{(0)} + b^{(1)}\right) + b^{(2)} = \underbrace{W^{(2)}W^{(1)}}_{:=W}x^{(0)} + \underbrace{W^{(2)}b^{(1)} + b^{(2)}}_{:=b} = Wx^{(0)} + b$$

Composition of Affine Linear Transformations



Classification With Affine Linear Transformations

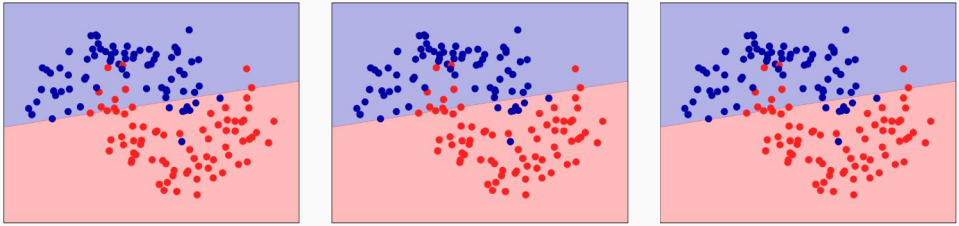
Here is a simple classification example:



1 neuron | 10 neurons | 100 neurons

Classification With Affine Linear Transformations

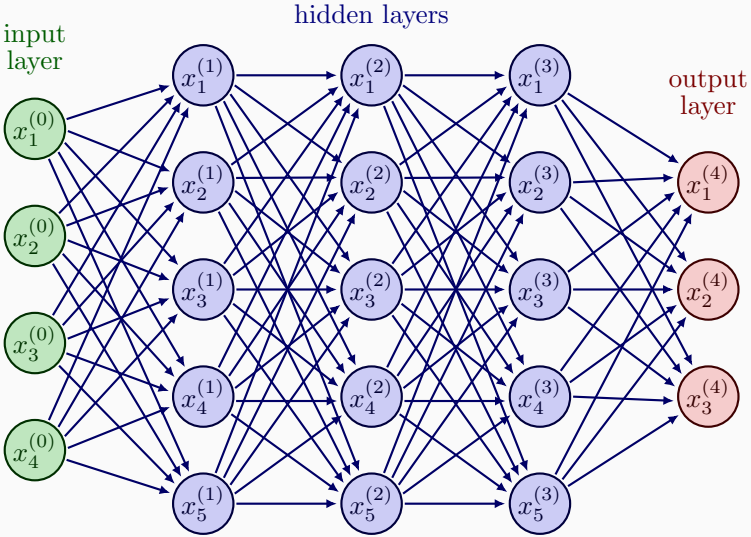
Here is a simple classification example:



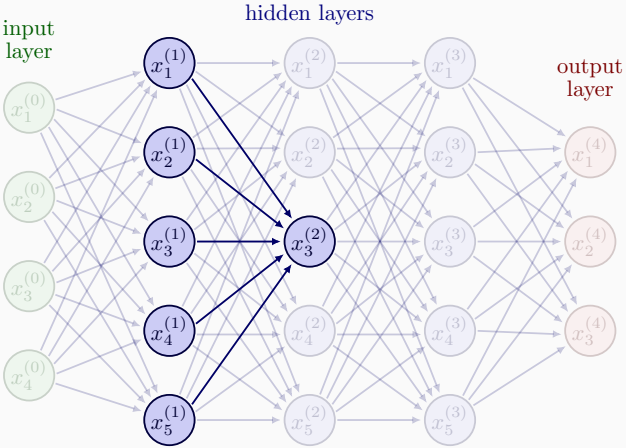
1 neuron | 10 neurons | 100 neurons

We have to modify the model to fit more complex functions.

Structure of an Artificial Neural Network Model (Part II)

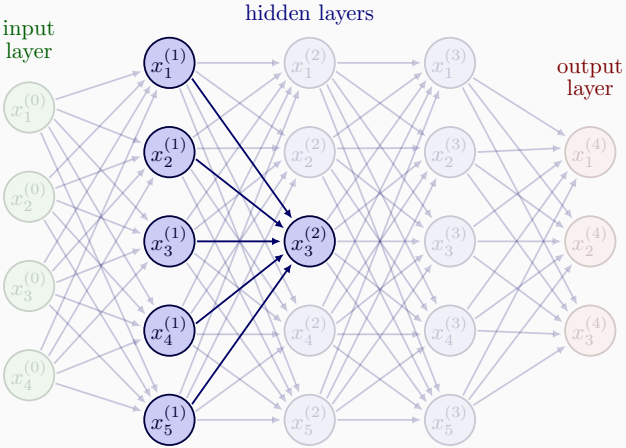


Structure of an Artificial Neural Network Model (Part II)



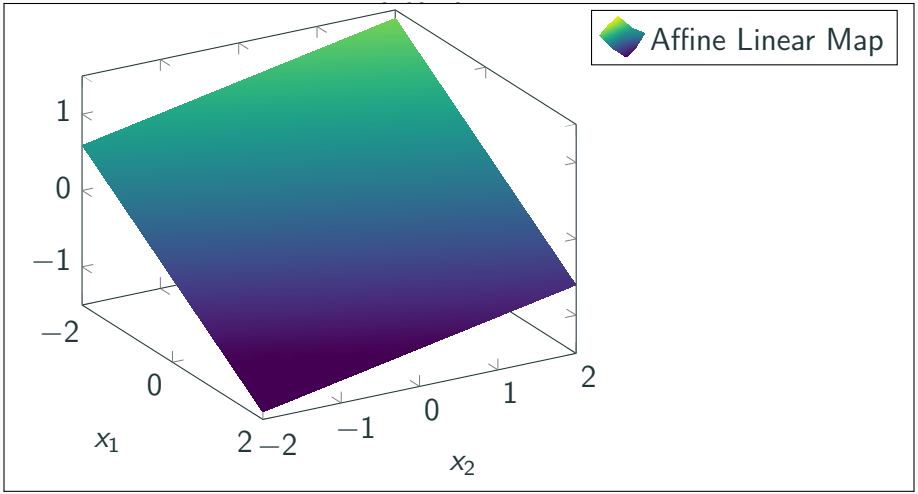
$$x_3^{(2)} = \sigma\Big(\sum_{i=1}^{5} w_i^{(2)} x_i^{(1)} + b_3^{(2)}\Big)$$

Structure of an Artificial Neural Network Model (Part II)



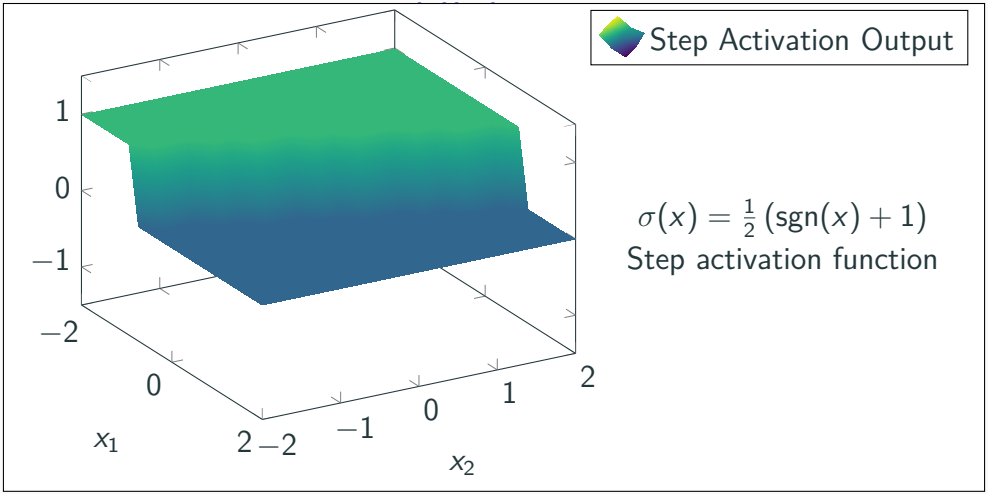
$$x_3^{(2)} = \underbrace{\sigma}_{\text{activation}} \left(\sum_{i=1}^{5} \underbrace{w_i^{(2)}}_{\text{weight}} x_i^{(1)} + \underbrace{b_3^{(2)}}_{\text{bias}} \right)$$

Structure of an Artificial Neural Network Model (Part II)



$$x_3^{(2)} = \sum_{i=1}^{5} \underbrace{w_i^{(2)}}_{\text{weight}} x_i^{(1)} + \underbrace{b_3^{(2)}}_{\text{bias}}$$

Structure of an Artificial Neural Network Model (Part II)

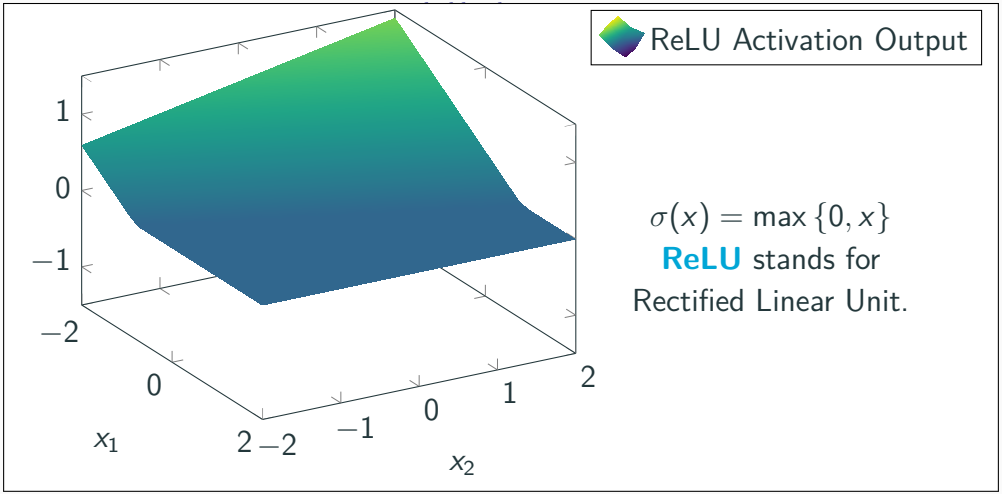


$$\sigma(x) = \frac{1}{2}\left(\operatorname{sgn}(x) + 1\right)$$

Step activation function

$$x_3^{(2)} = \underbrace{\sigma}_{\text{activation}} \left(\sum_{i=1}^{5} \underbrace{w_i^{(2)}}_{\text{weight}} x_i^{(1)} + \underbrace{b_3^{(2)}}_{\text{bias}} \right)$$

Structure of an Artificial Neural Network Model (Part II)

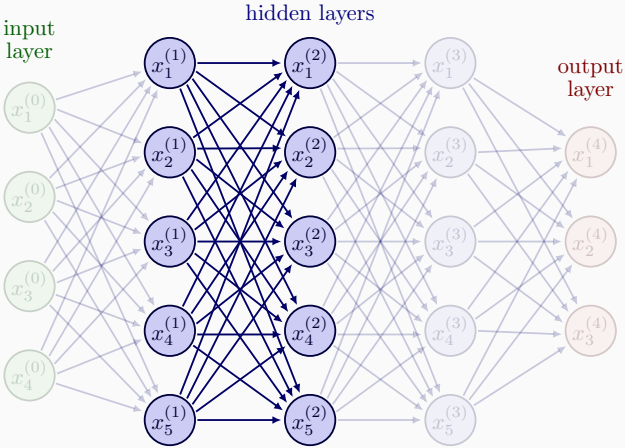


$\sigma(x) = \max\{0, x\}$

ReLU stands for Rectified Linear Unit.

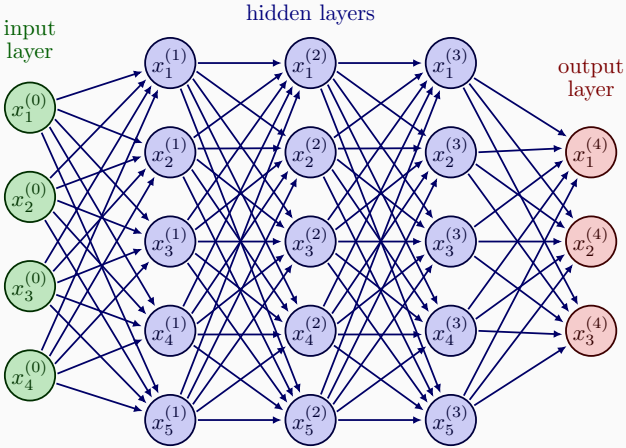
$$x_3^{(2)} = \underbrace{\sigma}_{\text{activation}} \left(\sum_{i=1}^{5} \underbrace{w_i^{(2)}}_{\text{weight}} x_i^{(1)} + \underbrace{b_3^{(2)}}_{\text{bias}} \right)$$

Structure of an Artificial Neural Network Model (Part II)



$$x^{(2)} = \sigma\left(W^{(2)}x^{(1)} + b^{(2)}\right)$$

Structure of an Artificial Neural Network Model (Part II)

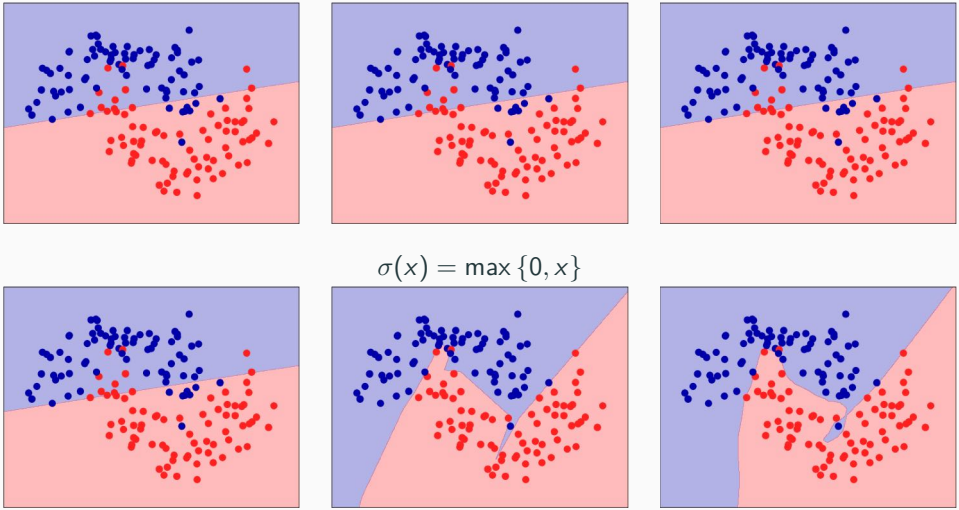


$$x^{(j+1)} = \sigma\left(W^{(j+1)}x^{(j)} + b^{(j+1)}\right)$$

The Power of Nonlinearity

Here is a simple classification example.

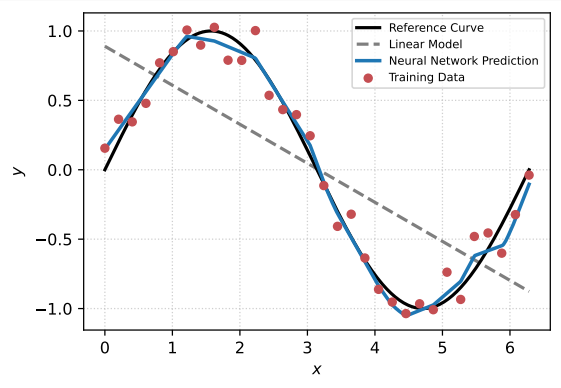
no σ

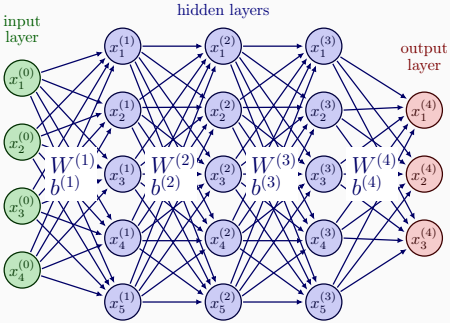


$\sigma(x) = \max\{0, x\}$

1 neuron | 10 neurons | 100 neurons

Training a Neural Network





Artificial neural network function:

$$\mathrm{ANN}(x)$$

Minimize

$$\sum_{i=1}^{n} (y_i - \mathrm{ANN}(x_i))^2$$

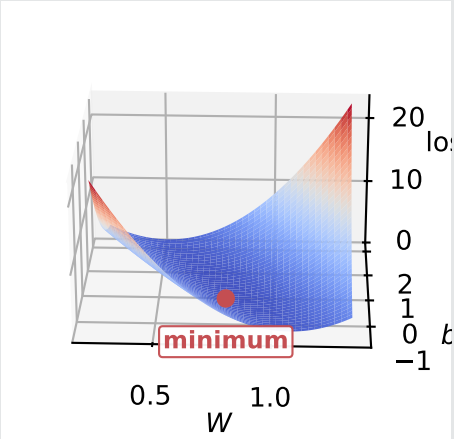
with respect to **weights** $W^{(j)}$ and **biases** $b^{(j)}$.

Fitting a Linear Versus a Neural Network Model

Fitting a Linear Model

Minimize

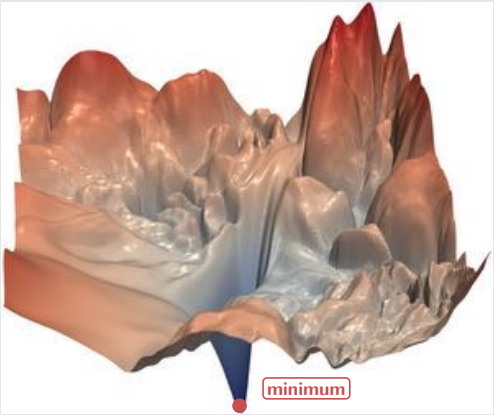
$$\sum_{i=1}^{n}(y_i - (Wx_i + b))^2.$$



Fitting a Neural Network Model

Minimize

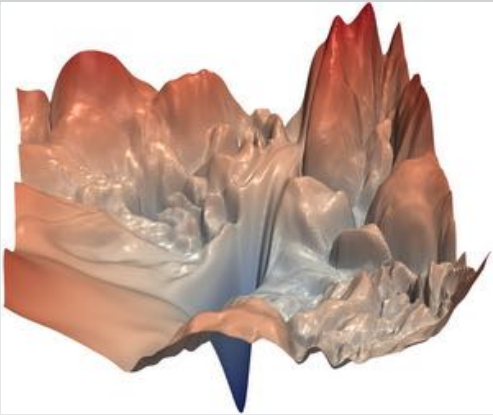
$$\sum_{i=1}^{n}(y_i - \mathrm{ANN}(x_i))^2.$$



Taken from https://github.com/tomgoldstein/loss-landscape

Rolling a Ball Down a Hill

Visualizing the Loss Landscape

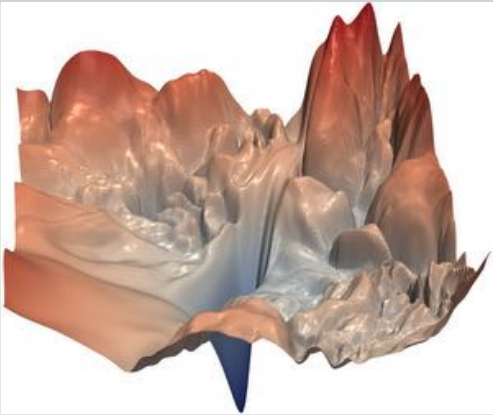


Taken from https://github.com/tomgoldstein/loss-landscape

Training Process

Rolling a Ball Down a Hill

Visualizing the Loss Landscape



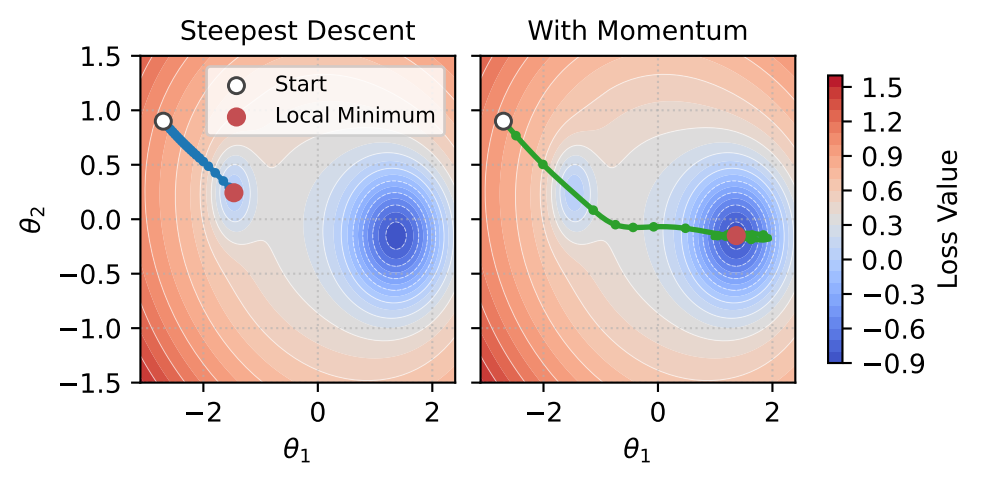
Taken from https://github.com/tomgoldstein/loss-landscape

- Move in the **steepest descent** direction

Training Process

- A bit of **momentum** can help the ball roll past shallow **local minima**

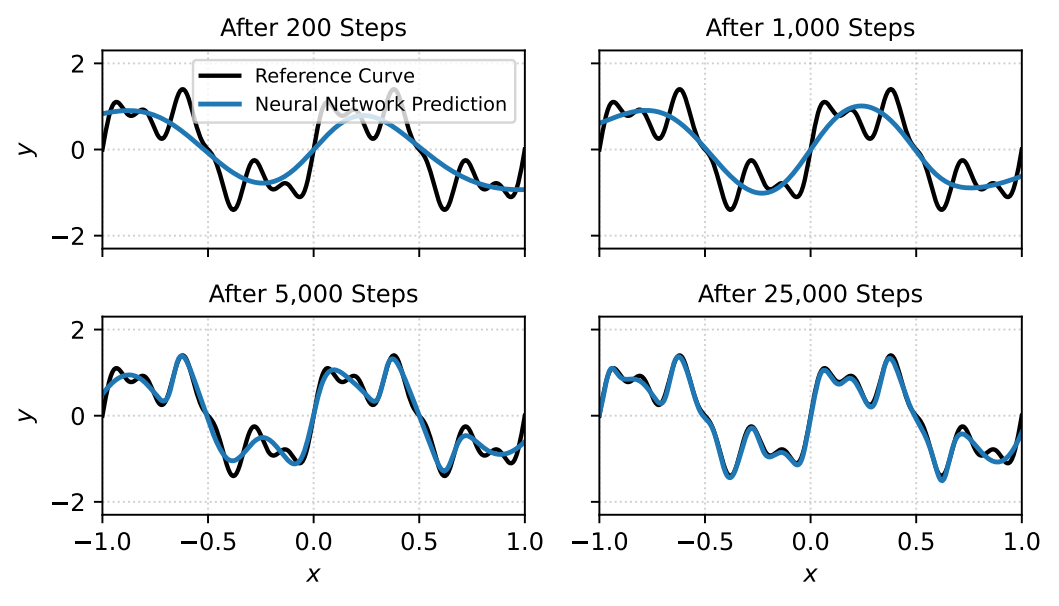
Why Rolling a Ball Is a Good Analogy?



These two panels show a key training idea:

- Steepest descent can get stuck.
- Momentum can help it move on.

Training Progress



Intermediate Summary

- An artificial neural network consists of layers: each applies a **linear transformation**

$$Wx + b$$

  followed by a **nonlinear activation**

$$\sigma$$

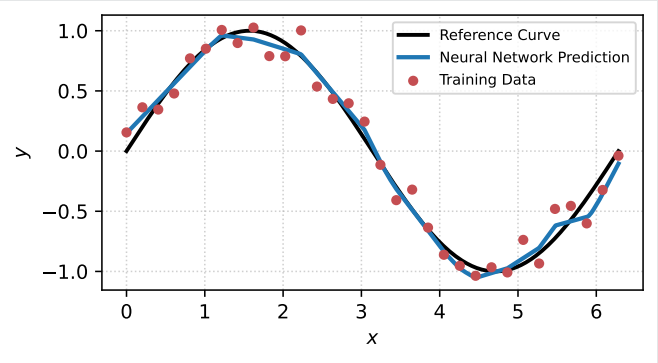
- **Nonlinearity** gives neural networks their power
- Training: **adjusting** W and b
- Training can be seen as rolling a ball down a hill

Neural Network Example in Python

Code Example

This example shows how a **neural network** can learn the **sine function** from **sample data**.

You can then change a few **parameters** yourself and see how the fit depends on the amount of **data**, the **network size**, and the **training time**.



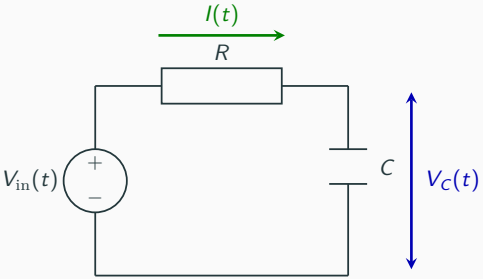
Try It Yourself

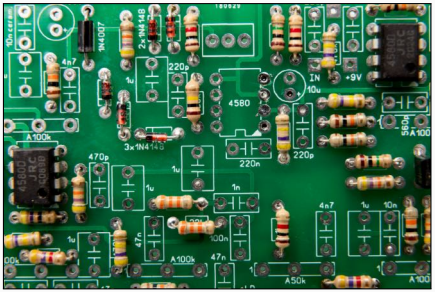
https://searhein.short.gy/nn

"'=latex **Hint**: the button ▶▶ runs all cells of the notebook. "'

3. The RC Circuit Model

The Physical Problem?





Source: freepik.com

What Is The Setting?

An RC circuit has three parts:

- a resistor R,
- a capacitor C,
- and an input voltage $V_{\text{in}}(t)$.

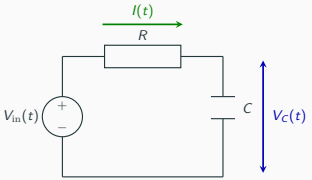
Main Question: How does the **capacitor voltage** $V_C(t)$ change in time?

Key Idea: the capacitor stores charge, so its voltage changes **gradually**.

Applications

- In **chargers** and **power supplies**, they help keep the voltage stable.
- In **audio systems**, they filter out unwanted **noise**.
- In **dimmers** and **LEDs**, they create smooth transitions.

From Circuit Laws to the RC Equation



Kirchhoff's voltage law

Around the loop, the voltage drops add up to the applied voltage:

$$V_{\text{in}}(t) = V_R(t) + V_C(t).$$

Component laws

$$V_R(t) = RI(t), \qquad I(t) = C\,\dot{V}_C(t).$$

First law: the voltage across the resistor equals resistance times current.
Second law: the capacitor current is proportional to how fast its voltage changes.

Substitute into **Kirchhoff's law**:

$$\begin{aligned} V_{\text{in}}(t) &= V_R(t) + V_C(t) = RI(t) + V_C(t) \\ &= RC\,\dot{V}_C(t) + V_C(t). \end{aligned}$$

So the unknown voltage $V_C(t)$ must satisfy

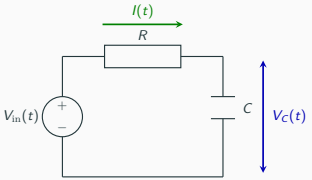
$$V_{\text{in}}(t) = RC\,\dot{V}_C(t) + V_C(t).$$

This is the main **RC differential equation**. It is convenient to introduce $\tau = RC$, so that the equation becomes

$$V_{\text{in}}(t) = \tau\,\dot{V}_C(t) + V_C(t).$$

Here $\dot{V}_C(t)$ is the rate of change of the capacitor voltage, and τ is the **time constant**: a larger τ means a slower response.

From Circuit Laws to the RC Equation



Kirchhoff's voltage law

Around the loop, the voltage drops add up to the applied voltage:

$$V_{\text{in}}(t) = V_R(t) + V_C(t).$$

Component laws

$$V_R(t) = RI(t), \qquad I(t) = C\,\dot{V}_C(t).$$

First law: the voltage across the resistor equals resistance times current.
Second law: the capacitor current is proportional to how fast its voltage changes.

Substitute into **Kirchhoff's law**:

$$\begin{aligned} V_{\text{in}}(t) &= V_R(t) + V_C(t) = RI(t) + V_C(t) \\ &= RC\,\dot{V}_C(t) + V_C(t). \end{aligned}$$

So the unknown voltage $V_C(t)$ must satisfy

$$V_{\text{in}}(t) = RC\,\dot{V}_C(t) + V_C(t).$$

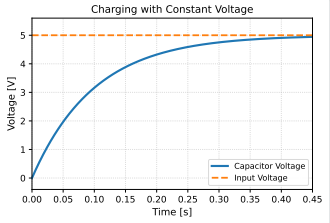
This is the main **RC differential equation**. It is convenient to introduce $\tau = RC$, so that the equation becomes

$$V_{\text{in}}(t) = \tau\,\dot{V}_C(t) + V_C(t).$$

Here $\dot{V}_C(t)$ is the rate of change of the capacitor voltage, and τ is the **time constant**: a larger τ means a slower response.

Three Basic RC Examples

1. Charging



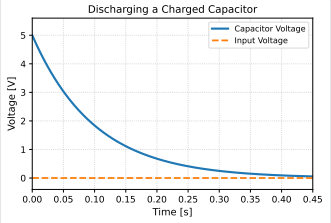
Input

$$V_{\text{in}}(t) = 5\,\text{V}$$

Initial value

$$V_C(0) = 0\,\text{V}$$

2. Discharging



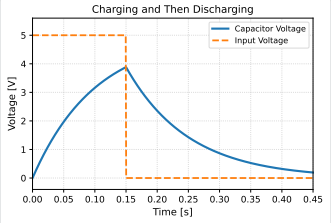
Input

$$V_{\text{in}}(t) = 0\,\text{V}$$

Initial value

$$V_C(0) = 5\,\text{V}$$

3. Switching



Input

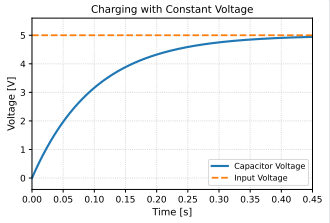
$$V_{\text{in}}(t) = \begin{cases} 5\,\text{V}, & 0 \le t < t_s, \\ 0\,\text{V}, & t \ge t_s \end{cases}$$

Initial value

$$V_C(0) = 0\,\text{V}$$

Three Basic RC Examples

1. Charging



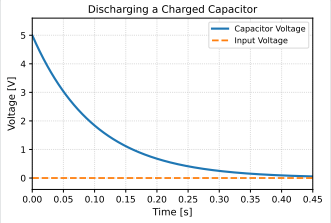
Input

$$V_{\text{in}}(t) = 5\,\text{V}$$

Initial value

$$V_C(0) = 0\,\text{V}$$

2. Discharging



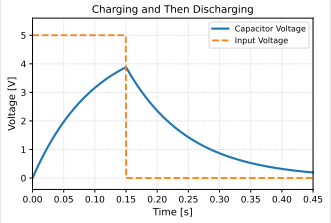
Input

$$V_{\text{in}}(t) = 0\,\text{V}$$

Initial value

$$V_C(0) = 5\,\text{V}$$

3. Switching



Input

$$V_{\text{in}}(t) = \begin{cases} 5\,\text{V}, & 0 \le t < t_s, \\ 0\,\text{V}, & t \ge t_s \end{cases}$$

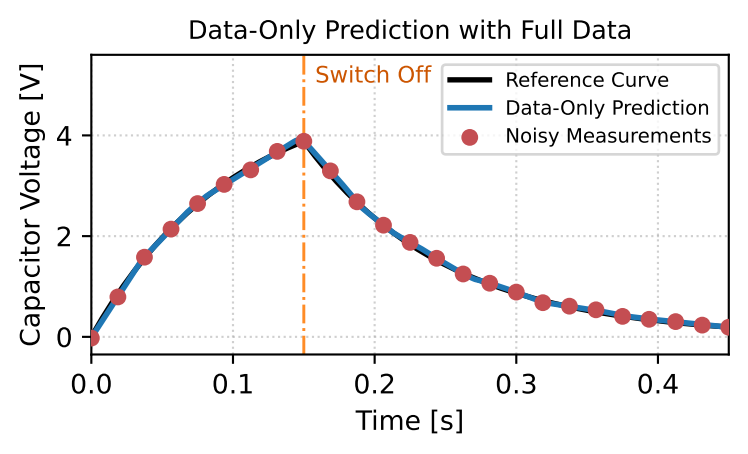
Initial value

$$V_C(0) = 0\,\text{V}$$

Next, we discuss how to approximate the solution of Example 3 using a neural network.

4. From Data Fitting to Physics-Informed Learning

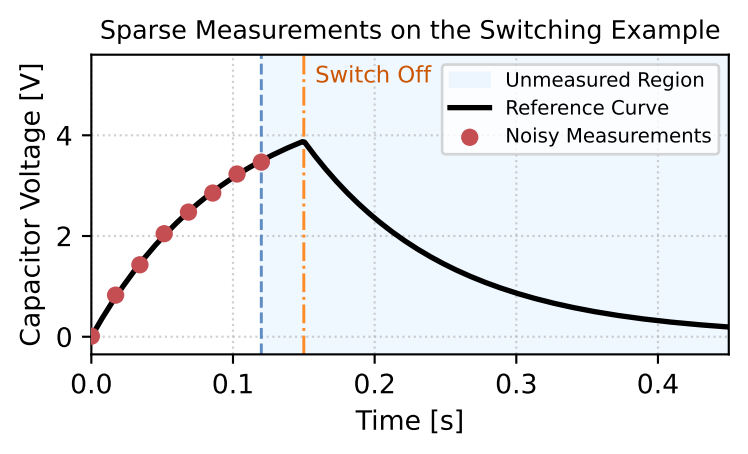
If Data Are Available Everywhere



Here, we assume that data for the full voltage curve is available. A neural network can learn the curve:

$$\mathcal{L}_{\text{data}}(\theta) = \frac{1}{N}\sum_{i=1}^{N}\left(V_i - \text{ANN}(t_i)\right)^2 .$$

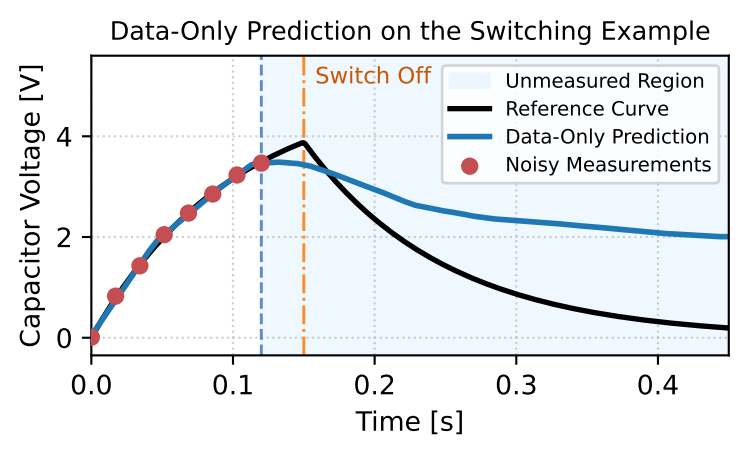
Now Remove Part of the Data



We only observe the **beginning**; later times are **not measured**.

Can the model predict where no data is available?

Data Only: Good Fit, Bad Prediction



Data-only

- Good fit where data is available
- Bad prediction outside the data range

Can Physics Help?

Idea: Use data where we have measurements, and use the RC equation where measurements are missing.

For the RC circuit,

$$\tau\,\dot{V}_C(t) + V_C(t) = V_{\text{in}}(t) \quad \Leftrightarrow \quad \tau\,\dot{V}_C(t) + V_C(t) - V_{\text{in}}(t) = 0$$

This is the **physics error**. Now, replace the unknown voltage by the network prediction $\mathrm{ANN}(t)$:

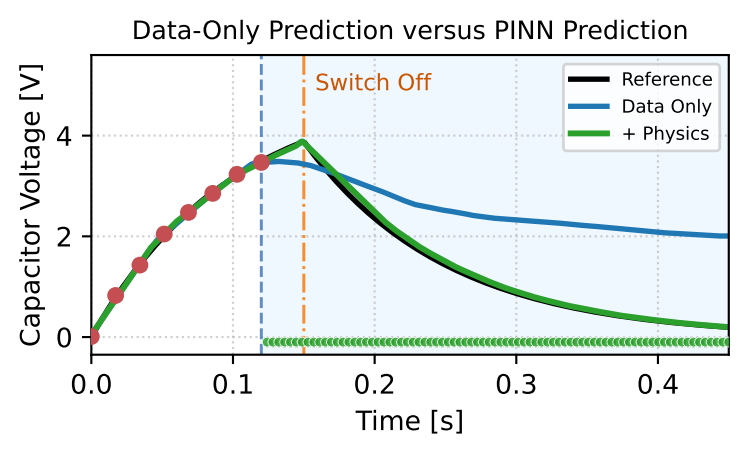
$$r(t) = \tau\,\dot{\mathrm{ANN}}(t) + \mathrm{ANN}(t) - V_{\text{in}}(t) \stackrel{!}{=} 0.$$

$$\text{Minimize} \qquad \lambda_{\text{data}}\,\frac{1}{N}\sum_{i=1}^{N}\left(V_i - \mathrm{ANN}(t_i)\right)^2 + \lambda_{\text{phys}}\,\frac{1}{M}\sum_{j=1}^{M} r_\theta(\tau_j)^2.$$

We call this a **physics-informed neural network (PINN)**:

Raissi, Perdikaris, and Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations,'' *Journal of Computational Physics*, 2019.

Adding Physics Improves the Prediction



Data-only

- Good fit where data is available
- Bad prediction outside the data range

Physics-informed

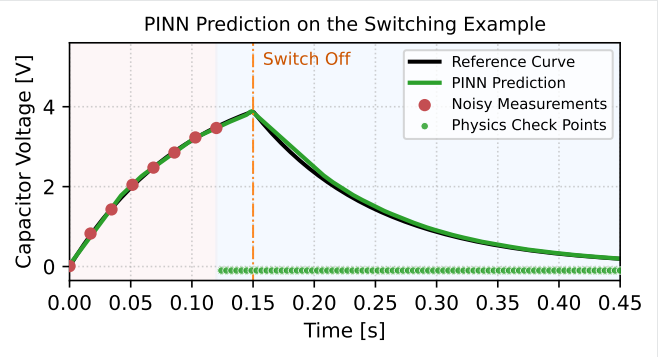
- Uses data where available and otherwise relies on physics
- Better prediction on the whole range

PINN Example in Python

Code Example

This example shows how a **PINN** combines sparse **RC data** with the **RC equation**.

You can then change a few **parameters** yourself and see how **data** and **physics** together improve the prediction.

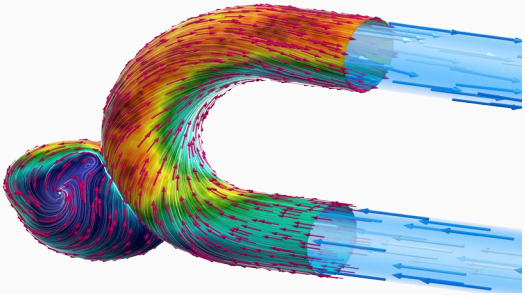


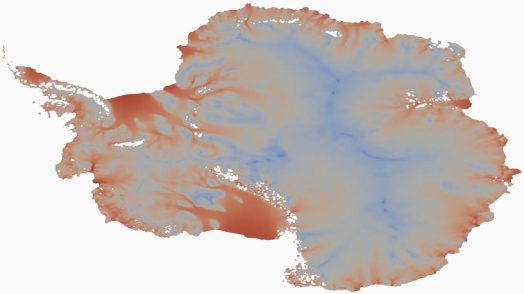
Try It Yourself

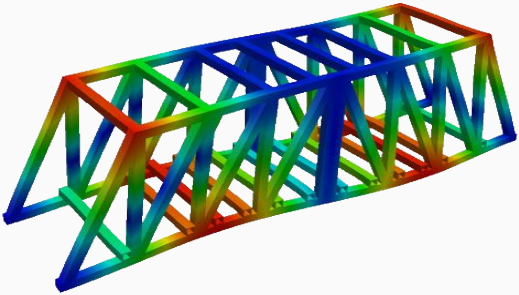
https://searhein.short.gy/pinn

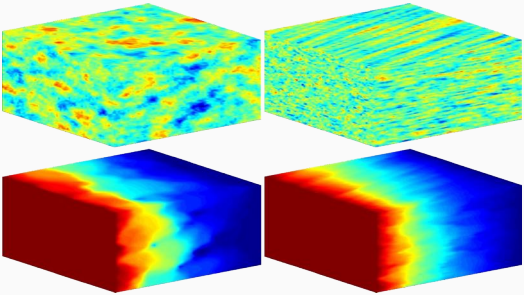
Hint: the button ▶▶ runs all cells of the notebook.

Where Else Does This Matter?









If You Want to Explore More

Deep Learning

- **Courses**
  - TU Delft: https://ocw.tudelft.nl/courses/ai-skills-introduction-to-unsupervised-deep-and-reinforcement-learning/subjects/module-4-introduction-to-deep-learning/
  - Stanford University: https://cs230.stanford.edu/

Physics-Informed Neural Networks

- **Blog Post**
  - Ben Moseley: So what is a physics-informed neural network? https://benmoseley.blog/my-research/so-what-is-a-physics-informed-neural-network/
- **Videos**
  - CAM Lab videos https://www.youtube.com/@CAMLabETHZurich/videos
  - Eigensteve videos https://www.youtube.com/@Eigensteve

Summary

What We Saw

- A **neural network** can learn a **curve from data**, and its **nonlinearity** makes it more flexible than a straight line.
- During **training**, we repeatedly adjust the model so its **predictions** match the data better.
- An **RC circuit** follows a simple **physical law** that produces **charging** and **discharging** curves.
- When data are **missing**, adding the **RC equation** can keep the prediction more **realistic**.

Learning Goals Revisited

You should now be able to:

- Describe what a **neural network** is and how it approximates complex functions,
- Solve the **RC-circuit equation**,
- Explain why **data + physics** can outperform data alone.

Main message: **when data are limited, combining data with physics can improve the prediction.**

Thank you for your attention!

Questions?

Feedback: `a.heinlein@tudelft.nl`

Repository

https://searhein.short.gy/repo